Experiment No: 07

Experiment Name:

Demonstration of Polymorphism using Method Overloading

Task 1

Task Name

Create a Calculator class that performs addition using Method Overloading.

Objective:

1. To understand the concept of method overloading in Java.
2. To perform addition using different types of parameters.
3. To implement compile-time polymorphism in Java programs.
4. To learn how the same method name can perform different operations.

Problem Analysis:

In Java, method overloading allows multiple methods with the same name but different parameters. This is an example of compile-time polymorphism. In this task, a class named Calculator is created where the method add() is overloaded to perform addition in different ways. One method adds two integers, another adds two floating-point numbers, and another adds three integers.

Input:

Integers and floating point numbers for addition.

Process:

Different versions of the add() method are created with different parameters. When the method is called, Java automatically selects the correct method depending on the arguments.

Output

The program prints the results of the addition operations.

Algorithm

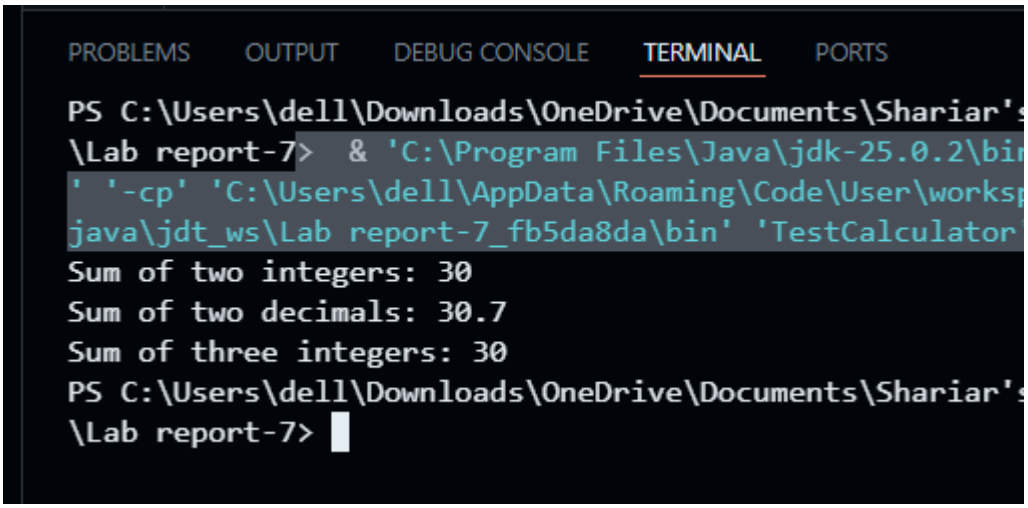
1. Start the program.
2. Create a class named Calculator.

3. Create a method add(int a, int b) to add two integers.
4. Create a method add(double a, double b) to add two decimal numbers.
5. Create a method add(int a, int b, int c) to add three integers.
6. Create the main method.
7. Create an object of the Calculator class.
8. Call the overloaded methods and print the results.
9. End the program.

Source Code:

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }
    double add(double a, double b) {
        return a + b;
    }
    int add(int a, int b, int c) {
        return a + b + c;
    }
}
public class TestCalculator {
    public static void main(String[] args) {
        Calculator c = new Calculator();
        System.out.println("Sum of two integers: " + c.add(10, 20));
        System.out.println("Sum of two decimals: " + c.add(10.5, 20.2));
        System.out.println("Sum of three integers: " + c.add(5, 10, 15));
    }
}
```

Output:



Discussion

This program demonstrates **method overloading**, where multiple methods have the same name but different parameters. The Java compiler determines which method to call based on the arguments provided during compilation. This technique improves code readability and allows the same method name to perform multiple tasks.

Task 2:

Task Name:

Create a Shape class to calculate area using Method Overloading

Objective:

1. To implement method overloading in Java.
2. To calculate the area of different shapes using the same method name.
3. To understand how parameter types determine method selection.
4. To demonstrate compile-time polymorphism.

Problem Analysis:

Method overloading can also be used to perform different calculations using the same method name. In this task, a class named Shape is created. The method area() is overloaded to calculate the area of different shapes. One method calculates the area of

a rectangle using length and width, while another method calculates the area of a circle using radius.

Input:

Length and width for rectangle, radius for circle.

Process:

Two methods with the same name area() are created with different parameters. The correct method is called depending on the provided arguments.

Output:

The program prints the area of the rectangle and circle.

Algorithm:

1. Start the program.
2. Create a class named Shape.
3. Create a method area(int length, int width) to calculate rectangle area.
4. Create another method area(double radius) to calculate circle area.
5. Create the main method.
6. Create an object of the Shape class.
7. Call the methods and display the results.
8. End the program.

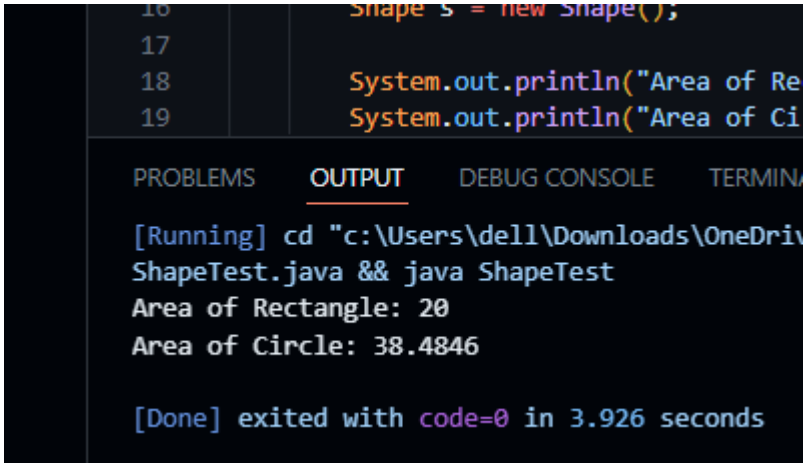
Source code:

```
class Shape {
   int area(int length, int width) {
      return length * width;
   }
   double area(double radius) {
      return 3.1416 * radius * radius;
   }
}
```

```
public class ShapeTest {


    public static void main(String[] args) {
        Shape s = new Shape();
        System.out.println("Area of Rectangle: " + s.area(5, 4));
        System.out.println("Area of Circle: " + s.area(3.5));
    }
}
```

Output:



Discussion:

This experiment shows how method overloading allows the same method name to perform different calculations. The area() method works for both rectangle and circle depending on the parameters passed. This makes the program more flexible and easier to maintain.